

10_mirror_haircut

July 3, 2026

1 10 — Lagged mirror, full coverage + haircut sweep (the trustworthy version)

Notebook 09 looked positive but was **not trustworthy**: the CLOB price-history endpoint returned nothing for ~83% of positions (resolved markets are largely delisted from it), so the result was computed on a self-selected 17% sliver. That's a hidden selection bias, and we don't build conclusions on 17% of the data.

The fix: we already hold the sharps' *own* transaction prices in their activity feed — the exact price they bought and sold at. So we reconstruct each round trip directly (100% coverage, no CLOB dependency), and model the cost of being a step behind as an explicit **haircut** H on both legs: - our entry = their average **buy** price $\times (1 + H)$ (we get in later / worse) - our exit = their average **sell** price $\times (1 - H)$ (we get out later / worse), or hold to resolution if they never sold.

Then we **sweep** H from 0 to 12% to find the *break-even latency cost* — how much friction the edge can absorb before it dies. That's far more informative than one guessed delay.

```
[1]: import importlib, pmc
importlib.reload(pmc)
from pmc import CFG, get_leaderboard, get_closed_positions, get_user_trades
import pandas as pd, numpy as np
print("full-coverage lagged mirror - reconstructing round trips from activity_
prices")
```

full-coverage lagged mirror - reconstructing round trips from activity prices

1.1 1. PIT-qualified positions (same as nb 08/09, cached)

```
[2]: cands = {}
for w in ("ALL", "MONTH"):
    for r in get_leaderboard(window=w, limit=CFG.WF_CANDIDATES):
        cands.setdefault(r["wallet"], r)
rows = []
for wallet in cands:
    for p in get_closed_positions(wallet, max_positions=600):
        cost = float(p.get("totalBought") or 0)
        if cost <= 0:
            continue
```

```

        rows.append({"wallet": wallet, "conditionId": p.get("conditionId"),
        ↪ "outcome": p.get("outcome"),
        ↪ "asset": p.get("asset"), "entry": float(p.get("avgPrice"))
        ↪ or 0),
        ↪ "entry_ts": int(p.get("timestamp") or 0), "endDate": p.
        ↪ get("endDate"),
        ↪ "realizedPnl": float(p.get("realizedPnl") or 0), "cost":
        ↪ cost,
        ↪ "won": 1 if float(p.get("realizedPnl") or 0) > 0 else 0})
h = pd.DataFrame(rows)
h["res_ts"] = (pd.to_datetime(h["endDate"], errors="coerce", utc=True).
        ↪ astype("int64") // 10**9)
h = h[(h["entry_ts"] > 0) & (h["res_ts"] > 0) & (h["res_ts"] > h["entry_ts"]) &
        ↪ (h["entry"] > 0)].copy()
h = h.sort_values("entry_ts").reset_index(drop=True)
qmask = np.zeros(len(h), dtype=bool)
for wallet, idx in h.groupby("wallet").groups.items():
    w = h.loc[idx]; rs = w.sort_values("res_ts")
    rt = rs["res_ts"].values; wc = np.cumsum(rs["won"].values)
    k = np.searchsorted(rt, w["entry_ts"].values, side="left")
    wins = np.where(k > 0, wc[np.clip(k - 1, 0, len(wc) - 1)], 0)
    wr = np.where(k > 0, wins / np.maximum(k, 1), 0.0)
    qmask[np.array(idx)] = (k >= CFG.WF_MIN_TRAILING_TRADES) & (wr >= CFG.
        ↪ WF_MIN_TRAILING_WINRATE)
q = h[qmask].copy()
q["backers"] = q.groupby(["conditionId", "outcome"])["wallet"].
        ↪ transform("nunique")
print(f"{len(q)} PIT-qualified positions")

```

4294 PIT-qualified positions

1.2 2. Reconstruct each round trip from activity PRICES (full coverage)

For each wallet we pull **only the sampled markets' trades** (activity filtered by conditionId), so we get the exact buys and sells for every position — ~100% coverage, no pagination-ceiling truncation. We volume-weight their buy and sell prices. No CLOB history.

```

[3]: sample = q.sample(n=min(CFG.MIRROR_SAMPLE, len(q)), random_state=42).copy()
by_wallet = sample.groupby("wallet")["conditionId"].apply(lambda s:
        ↪ sorted(set(s)).to_dict())
print(f"pulling market-filtered activity for {len(by_wallet)} wallets...")

# (wallet, asset) -> vwap buy/sell prices + sizes
acc = {}
for j, (wal, mkts) in enumerate(by_wallet.items()):
    for i in range(0, len(mkts), 3):
        ↪ # small chunks + short
        ↪ URLs, no 408 timeouts

```

```

    for t in get_user_trades(wal, markets=mkts[i:i + 3]):
        key = (wal, t.get("asset"))
        a = acc.setdefault(key, {"bq": 0.0, "bqp": 0.0, "sq": 0.0, "sqp": 0.
↪0, "last_sell": 0})
        sz = float(t.get("size") or 0); pr = float(t.get("price") or 0)
        if t.get("side") == "BUY":
            a["bq"] += sz; a["bqp"] += sz * pr
        elif t.get("side") == "SELL":
            a["sq"] += sz; a["sqp"] += sz * pr
            a["last_sell"] = max(a["last_sell"], int(t.get("timestamp") or 0))
↪0))
    if (j + 1) % 15 == 0:
        print(f" {j+1}/{len(by_wallet)} wallets")

def legs(wal, asset):
    a = acc.get((wal, asset))
    if not a or a["bq"] <= 0:
        return None
    buy_vwap = a["bqp"] / a["bq"]
    sold = a["sq"] >= 0.5 * a["bq"]
    sell_vwap = (a["sqp"] / a["sq"]) if a["sq"] > 0 else None
    return {"buy_vwap": buy_vwap, "sold": sold, "sell_vwap": sell_vwap}

recon = sample.apply(lambda r: legs(r["wallet"], r["asset"]), axis=1)
sample = sample.assign(**pd.json_normalize(recon).set_index(sample.index))
cov = sample["buy_vwap"].notna().mean()
print(f"coverage: {sample['buy_vwap'].notna().sum()}/{len(sample)} = {cov*100:.
↪0f}% (target: near 100%)")
# sanity: their reconstructed buy vwap should track the closed-position entry
↪price
chk = sample.dropna(subset=["buy_vwap"])
print(f"sanity - mean |buy_vwap - entry| = {(chk['buy_vwap'] - chk['entry']).
↪abs().mean():.3f} (should be small)")

```

pulling market-filtered activity for 53 wallets...

15/53 wallets

30/53 wallets

45/53 wallets

coverage: 569/600 = 95% (target: near 100%)

sanity - mean |buy_vwap - entry| = 0.016 (should be small)

1.3 3. Sweep the latency haircut

For each haircut H , the copier's per-position return is computed on the full sample. We report the **median** return and % **profitable** by backer count, and flag the break-even H .

```
[4]: def copy_ret(row, H):
    if pd.isna(row["buy_vwap"]):
        return np.nan
    buy = row["buy_vwap"] * (1 + H)
    if buy <= 0 or buy >= 1:
        return np.nan
    if row["sold"] and pd.notna(row["sell_vwap"]):
        sell = row["sell_vwap"] * (1 - H)
    else:
        sell = float(row["won"])          # held to resolution
    return (sell - buy) / buy

table = []
for H in [0.0, 0.02, 0.05, 0.08, 0.12]:
    r = sample.apply(lambda x: copy_ret(x, H), axis=1)
    for n in [1, 2, 3]:
        mask = sample["backers"] >= n
        s = r[mask].dropna()
        if len(s) == 0:
            continue
        table.append({"haircut": H, "min_backers": n, "n": len(s),
                      "median_ret": round(s.median(), 3),
                      "%_profitable": round((s > 0).mean(), 3)})
res = pd.DataFrame(table)
print("LAGGED MIRROR - median return by latency haircut and backer count:")
res.pivot(index="haircut", columns="min_backers", values="median_ret")
```

LAGGED MIRROR - median return by latency haircut and backer count:

```
[4]: min_backers      1      2      3
haircut
0.00      0.098  0.175  0.132
0.02      0.120  0.218  0.123
0.05      0.111  0.183  0.074
0.08      0.107  0.184  0.037
0.12      0.100  0.180  0.000
```

```
[5]: # same view, % profitable
res.pivot(index="haircut", columns="min_backers", values="%_profitable")
```

```
[5]: min_backers      1      2      3
haircut
0.00      0.684  0.706  0.707
0.02      0.620  0.656  0.671
0.05      0.591  0.632  0.618
0.08      0.564  0.598  0.548
0.12      0.553  0.579  0.507
```

1.4 4. The decision — read the haircut curve

- **Median stays clearly > 0 out to a realistic haircut (say 5–8%)** → the exit edge is robust to latency. That is a genuine, tradeable strategy. Next: raise `MIRROR_SAMPLE` to tighten it, then **paper-trade** before any capital.
- **Median crosses to 0 at a tiny haircut (2%)** → the edge only exists if you copy almost instantly and frictionlessly; in practice it's gone. That's the wall.

The **break-even haircut** is the headline: it is literally “how many cents of round-trip slippage the edge can survive.” A robust strategy survives several cents; a mirage dies at the first.

1.4.1 Trust checklist for this run

- **Coverage** (cell 2) should be near 100%. If it's low, the activity pull was truncated — raise the pagination cap or `MIRROR_SAMPLE`.
- **Sanity** (cell 2): reconstructed `buy_vwap` should be close to the closed-position entry price. A large gap means the round-trip reconstruction is off.
- Residual **survivorship** (today's leaderboard) and **equal-weighting** still apply, both nudging results slightly optimistic.
- Small `n` at backers 3 stays noisy — weigh the 1 and 2 columns more.